

S4-F — Dokumentacja techniczna

Contents

S4-F — Dokumentacja techniczna	1
Spis treści	2
1. Przegląd systemu	2
2. Układ repozytorium	3
3. Referencja modułów	3
4. Specyfikacja obserwacji	5
5. Pipeline akcji i wspomaganie kierowcy	6
6. Funkcja nagrody	6
7. Zakończenie epizodu	7
8. Architektury sieci	7
9. Integracja z TORCS (Windows i Linux)	8
10. Pipeline'y treningowe z komendami	9
11. Rozproszona infrastruktura Optun	10
12. Checkpointy i artefakty	11
13. Referencja konfiguracji	11
14. Ograniczenia i pułapki	12
15. Rozwiązywanie problemów	12

S4-F — Dokumentacja techniczna

English version: [TECHNICAL_DOCUMENTATION.md](#)

Sieciowy agent wyścigowy do TORCS/SCR (IBM AI Racing League), tor Corkscrew. To jest kompletna referencja techniczna repozytorium: architektura systemu, mapa modułów, specyfikacja obserwacji/akcji/nagrody, integracja z TORCS na Windows i Linux, pełny pipeline treningowy z komendami, formaty checkpointów oraz rozwiązywanie problemów.

Dokumenty towarzyszące:

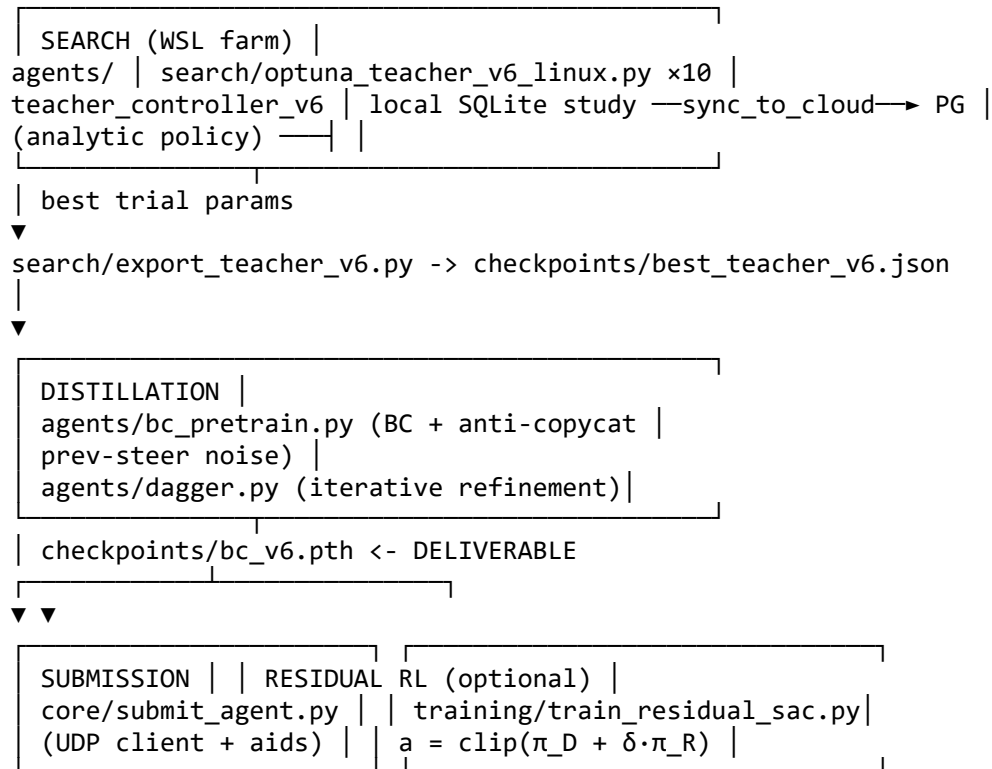
Dokument	Zawartość
case_study.pdf	Pełne inżynierskie case study (metodologia, eksperymenty, wyniki)
EXPERIMENT_LOG.md	Chronologiczny log inżynierski (surowy, nieedytowany)
physics_desync_and_variance_collapse_bug_2026-06-30.md	Post-mortem kaskady awarii residual-RL
DISTILL_COMMANDS.md	Gotowy przepis (copy-paste) na pipeline destylacji
README_WSL.md	Przewodnik budowy headless TORCS w WSL (oryginalne notatki)

Spis treści

1. Przegląd systemu
 2. Układ repozytorium
 3. Referencja modułów
 4. Specyfikacja obserwacji
 5. Pipeline akcji i wspomaganie kierowcy
 6. Funkcja nagrody
 7. Zakończenie epizodu
 8. Architektury sieci
 9. Integracja z TORCS (Windows i Linux)
 10. Pipeline'y treningowe z komendami
 11. Rozproszona infrastruktura Optuny
 12. Checkpointy i artefakty
 13. Referencja konfiguracji
 14. Ograniczenia i pułapki
 15. Rozwiązywanie problemów
-

1. Przegląd systemu

System tworzy sieciowego kierowcę poprzez **pipeline teacher-student**: analityczny, regułowy kontroler wyścigowy („teacher”) z 54 strojonymi parametrami jest optymalizowany rozproszonym przeszukiwaniem Optuny, a następnie kompresowany („destylowany”) do małego perceptronu wielowarstwowego przez klonowanie behawioralne (BC) i DAgger. Uczenie ze wzmocnieniem bez modelu (Soft Actor-Critic) pełni dwie role pomocnicze: *baseline’y* trenowane od zera oraz *residualne* dostrajanie zamrożonej, zdestylowanej polityki.



Interfejs runtime: serwer SCR wewnątrz TORCS wymienia pakiety UDP z `core/snakeoil3_gym.py` z częstotliwością 50 Hz (20 ms na takt, port 3001 + indeks sterownika). Każdy konsument buduje obserwacje przez `core/observation_utils.py` i przetwarza akcje przez `core/driving_aids.py`, więc zachowanie w treningu

i przy zgłoszeniu jest **identyczne z założenia**.

Główny wynik: checkpoints/bc_v6.pth (108 674 parametry, 429 KB) uzyskał okrążenie **92,04 s** ze startu zatrzymanego na Corkscrew w warunkach zgłoszeniowych (paliwo + uszkodzenia włączone), prędkość maksymalna 250 km/h. Residualna polityka uczenia ze wzmocnieniem na tej samej zamrożonej bazie (checkpoints/residual_sac_latest.zip) osiąga porównywalne okrążenie **~92 s**, więc wynik utrzymuje się pod RL, a nie tylko pod imitacją — oddajemy obu agentów.

2. Układ repozytorium

S4-F/

- README.md # przegląd projektu + szybki start
- S4-F_requirements.txt # przypięte zależności Pythona
- config.py # <- jedyne źródło prawdy dla CAŁEJ konfiguracji
- practice.xml # szablon konfiguracji wyścigu (MUSI zostać w rootcie, §14)
- autostart_win.py # auto-start wyścigu w GUI Windows (MUSI zostać w rootcie, §14)
- .gitignore

- core/ # runtime: łańcuch zgłoszenia + środowisko treningowe
- agents/ # analityczni teacherzy (v1-v6) + BC / DAgger / BC-anchored SAC
- search/ # rozproszona infrastruktura Optuny (wszystkie wersje)
- training/ # sterowniki treningu SAC / residual-SAC
- tools/ # diagnostyka, ewaluacja, analiza telemetrii
- phase2/ # pipeline etapu 2 (multi-car / mass-start)

- checkpoints/ # artefakty modeli (bc_v6.pth = deliverable)
- docs/ # ten plik, case study, log eksperymentów, przewodniki
- scripts/ # dawne launchery .bat dla Windows (proweniencja)

Moduły to **pakiety** Pythona: punkty wejścia uruchamiaj z katalogu głównego repo komendą `python -m <paket>.<modul>`, np. `python -m core.submit_agent`. (`config.py` leży w rootcie, więc `import config` rozwiązuje się dla każdego punktu wejścia startowanego z roota; zob. §14.)

3. Referencja modułów

3.1 core/ — runtime (łańcuch zgłoszenia + środowisko)

Moduł	Rola
core/submit_agent.py	Punkt wejścia konkursu: ładuje wagi BCNetwork, jeździ przez UDP z driving-aids
core/snakeoil3_gym.py	Klient UDP SCR: połączenie, parsowanie telemetrii, formatowanie komend, restart TORCS
core/observation_utils.py	surowy słownik telemetrii -> znormalizowany wektor 32/68-wym. float32
core/driving_aids.py	akcja sieci -> komenda TORCS: harmonogram biegów wg RPM, TCS, centrowanie na starcie
core/torcs_env_sac.py	środowisko Gymnasium: start TORCS (GUI Windows / Linux headless), pętla kroków UDP, terminacja, podpięcie nagrody
core/reward_functions.py	obliczanie nagrody etap 1 / etap 2 (§6)
core/custom_policy.py	LayerNormSACPolicy — aktor/krytyk SAC z LayerNorm po każdej warstwie ukrytej
core/telemetry_recorder.py	CSV auta co takt + CSV statystyk treningu co 100 kroków

Moduł	Rola
core/callbacks.py	callbacks SB3: checkpointy, zanik entropii, zamrożenie aktora

3.2 agents/ — polityki

Moduł	Model	Status
agents/teacher_controller_v6.py	rozprężnięta obwiednia hamowania + linia out-in-out + zmiana biegów przy niemal odcięciu	bieżący teacher
agents/teacher_controller_v5.py	6 bez linii wyścigowej	zastąpiony
agents/teacher_controller_v4.py	$\sqrt{\text{reach}}$ (sprężone proste/zakręty)	dawny
agents/teacher_controller_v3.py	12+12 statycznych waypointów; zawiera też evaluate_teacher używany przez search	dawny, ale nośny
agents/teacher_controller_v2.py	warianty lookahead PID	dawne
teacher_controller.py		
agents/tune_teacher.py	oprzyrządowanie Optuna v1/v2 + I/O parametrów JSON	dawne
agents/bc_pretrain.py	zbieranie rolloutów teachera + BCNetwork + trening BC z ważonym MSE i szumem prev-steer	bieżący
agents/dagger.py	pętla DAgger (agregaty etykietowane przez teachera, zasiane demonstracjami)	bieżący
agents/bc_anchored_sac.py	BCAnchoredSAC — podklasa SAC z kotwicą w stylu TD3+BC (tylko średnia)	bieżący

Kontrakt interfejsu teachera: `TeacherController(params).act(raw_obs) -> [steer, accel_brake]`, `get_gear() -> int`; per wersja: `sample_params(trial), params_from_optuna(dict)`.

3.3 search/ — rozproszona farma Optuna

Moduł	Rola
search/optuna_teacher_linux.py	wspólna hydraulika: <code>install_practice_xml(port)</code> , <code>headless launch_torcs (torcs -r)</code> , zawężony <code>kill_torcs</code>
search/optuna_teacher_v6_linux.py	pętla triali per worker dla v6 (<code>ask -> wyścig -> tell</code>), detekcja zaklinowania
search/optuna_teacher_v{2,4,5}_linux.py	to samo dla starszych kontrolerów (dawne)
search/optuna_teacher_v3.py	<code>make_storage()</code> (WAL SQLite / pulowany PostgreSQL) — używany przez każdy sterownik ; oprzyrządowanie triali v3
search/optuna_teacher_single.py	sweep na jednej maszynie Windows (dawny)
search/new_study_v6.py (_v2/_v4/_v5, new_study.py)	utworzenie study + zakolejkowanie triali zasiewowych
search/export_teacher_v6.py	najlepszy trial -> <code>checkpoints/best_teacher_v6.json</code>

Moduł	Rola
search/sync_to_cloud.py search/monitor.py search/pg_setup.py, pg_new_study.py, inspect_pg.py, save_v2_best.py	jednokierunkowe lustro lokalny-SQLite -> PostgreSQL dashboard farmy na żywo w terminalu administracja PostgreSQL / narzędzia study

3.4 training/ — sterowniki RL

Moduł	Rola
training/train_residual_sac.py	residual SAC na zamrożonej bazie: zasiew residuałem = 0, floor checks, tryb --eval. --resume none
training/residual_env.py	wymagane przy zmianie bazy ResidualTorcsEnv: opakowuje TorcsSACEnv, trzyma zamrożoną bazę, stosuje $\text{clip}(\pi_D + \delta \cdot \pi_R, -1, 1)$
training/train_sac.py	SAC od zera / dostrajanie, curriculum dwuetapowe, transfer_stage1_to_stage2 (chirurgia wag 32->68)
training/train_sac_v2.py training/optuna_sac.py, optuna_residual.py training/multi_instance_torcs.py	SAC od zera z curriculum limitu prędkości (dawny) HPO nad hiperparametrami SAC / residual (dawne) menedżer wielu instancji TORCS na Windows, porty 3001–3006 (era dawna)

3.5 tools/ — diagnostyka

Moduł	Rola
tools/eval_policy.py	ewaluacja polityki .pth w pętli zamkniętej (deterministyczna/stochastyczna)
tools/telemetry_lap.py	profil prędkość–dystans najlepszych parametrów study (narzędzie, które znalazło sufit 174 km/h)
tools/diag_lap.py	kontrola pojedynczego okrążenia / graficzna powtórka --watch
tools/inspect_best.py tools/gen_scr_server.py tools/granite_analysis.py	wypisanie metadanych najlepszego checkpointu generacja XML konfiguracji serwera SCR komentarz telemetry per zakręt przez LLM IBM Granite (opcjonalne; lokalny Ollama lub watsonx)

3.6 phase2/ — pipeline etapu 2 (multi-car)

Samodzielny wariant mass-start: race_config.py (generacja XML siatki przeciwników), residual_env_stage2.py (obserwacje 68-wym.), train_mass_start.py, submit_agent_stage2.py. Nie jest częścią punktowanego deliverable'a z próby czasowej.

4. Specyfikacja obserwacji

Etap 1 (próba czasowa): 32 wymiary. Etap 2 dokłada 36 dalmierzy przeciwników (/200 m) -> 68 wymiarów; indeksy 0–31 są ścisłym prefiksem, więc wagi z etapu 1 przenoszą się bezstratnie (training/train_sac.py::transfer_stage1_to_s

Indeks	Sygnał	Zakres surowy	Normalizacja
0–18	dalmierze krawędzi toru $d_1 \dots d_{19}$ (kąty wiązek $-45^\circ \dots +45^\circ$)	0–200 m	/200, clip [0, 1]
19, 20	speedX, speedY	± 300 km/h	/300
21	błąd kursu ψ względem stycznej toru	$[-\pi, \pi]$ rad	/ π
22	trackPos p (0 = środek, ± 1 = krawędzie)	$\approx [-1,5, 1,5]$	clip $[-1, 1]$
23	obroty silnika	0–18 700 rpm	/10 000, clip [0, 1]
24	bieg	1–6 (zakres SCR)	/7 (stała treningowa)
25–28	prędkości kątowe kół (FL, FR, RL, RR)	0–100 rad/s	/100
29	postęp okrążenia distFromStart	0–3 602 m	/3 602
30	bieżący czas okrążenia	0–120 s	/120
31	poprzednia komenda skrętu	$[-1, 1]$	tożsamość

Uwaga o indeksie 31. prev_steer utrzymuje proces markowowskim przy dynamice skrętu warstwy aids, ale umożliwia *skrót copycat* w klonowaniu behawioralnym (steer $\approx$ prev_steer, Codevilla i in. 2019). agents/bc_pretrain.py::train_bc dodaje więc szum gaussowski ($\sigma = 0,15$, obcięty) do tej cechy w każdym minibatchu treningowym. Obserwacje w czasie wdrożenia pozostają niezmienione.

5. Pipeline akcji i wspomaganie kierowcy

```

NN output [steer, accel_brake] → [-1, 1]2
-> core/driving_aids.apply_aids():
1. steering-rate limiter (disabled by default)
2. launch centering (v < 40 km/h AND lap time < 8 s:
steer = clip(0.30*trackPos, ±0.4))
3. accel/brake split (accel_brake ≥ 0 -> throttle,
< 0 -> brake = |accel_brake|)
+ TCS: if (rear - front) wheel-spin > 5 rad/s and v ≥ 30 km/h,
throttle *= max(0.1, 5/slip)
4. automatic gearbox (pure RPM: up > 17,800 rpm & g < 6;
down < 9,000 rpm & g > 1)
-> TORCS command {steer, accel, brake, gear}

```

Uwaga o zakresie biegów. Protokół SCR przyjmuje komendy biegów **tylko –1...6**; core/snakeoil3_gym.py resetuje każdą inną wartość do **luzu**. Definicja auta car1-ow1.xml wymienia siódme przełożenie do przodu, ale jest ono nieosiągalne przez SCR. Telemetria z 2,17 mln zarejestrowanych taktów potwierdza, że auto osiąga prędkość maksymalną ~255 km/h na **piątym** biegu przy ~17 200 rpm (szósty jest używany na <0,1 % taktów); decydującą poprawką był *próg* zmiany biegu, a nie ich liczba.

Ta sama funkcja działa w treningu (core/torcs_env_sac.step), przy zbieraniu danych teachera, ewaluacji i zgłoszeniu — progi zmiany biegów i TCS nigdy nie mogą rozjechać się między zbieraniem danych a wdrożeniem. To miało znaczenie: wcześniejszy harmonogram zbyt wczesnej zmiany przy 8 200 rpm w tej warstwie ograniczał każdego agenta do 174 km/h (zob. docs/EXPERIMENT_LOG.md §8 oraz case study).

6. Funkcja nagrody

Etap 1 (core/reward_functions.py::reward_stage1), na takt 20 ms:

```
r = 1.0 · Δ distRaced # progress [m]; clamped to [0, 50]
```

```

- 0.1 # per-step time cost
- 0.05 · trackPos2 # centring (quadratic)
- 0.05 · |angle| # heading alignment
- 0.1 · |Δ steer| # steering smoothness
+ 0.2 · (speedX/300) if min(track[8:11]) > 150 # straight-line bonus
+ 1.0 · (speedX/300) · |cos ψ| · (1 - |ρ|) · sharp # corner-speed reward
- 0.025 · min(Δ damage, 200) # damage
+ events

```

gdzie sharp = $\max(0, 1 - \min(\text{track}[7:12])/150)$, a zdarzenia to: wypadnięcie z toru -1 na krok, jazda tyłem -10, ukończenie okrążenia +500 + $\max(0, 90 - \text{lap_time})$. Etap 2 dodaje karę za bliskość przeciwnika (liniowa poniżej 10 m), +5 za wyprzedzenie, -5 za kontakt.

Uwaga projektowa: wcześniejszą nagrodę $\text{speedX} \cdot \cos(\psi)$ porzucono w S2 — jej cała to również tylko dystans, więc pełzanie i ściganie dają podobny zwrot na stałym horyzoncie. Forma progress + koszt czasu + bonus za okrążenie czyni czas okrążenia jawnym w zwrocie.

7. Zakończenie epizodu

Z core/torcs_env_sac.py (progi w config.TorcsConfig):

Warunek	Próg
poza torem	$ \text{trackPos} > 1,10$ przez > 30 kolejnych taktów (0,6 s tolerancji — tarki dozwolone)
jazda tyłem	$\cos(\text{angle}) < 0$
uszkodzenia	skumulowane > 5 000
zaklinowanie	postęp do przodu < 2 m przez 250 taktów
timeout	9 000 taktów (180 s)
połączenie	cisza UDP / śmierć procesu TORCS

Nie przywracaj klauzuli $\min(\text{track}) < 0$ do predykatu wypadnięcia z toru: skośne dalmierze zgodnie z prawdą zwracają -1 (brak powierzchni w zasięgu 200 m) w długich, ciasnych zakrętach. Ta właśnie klauzula po cichu uciniała epizody treningowe na nawrotce ~2 400 m dla każdej generacji agenta, dopóki jej nie usunięto (udokumentowany defekt; zob. case study §5.4.3).

8. Architektury sieci

8.1 BCNetwork (deliverable)

Zdefiniowana w agents/bc_pretrain.py; wiernie odzwierciedla trzon aktora SAC:

```

input 32 -> Linear 256 -> LayerNorm -> ReLU
-> Linear 256 -> LayerNorm -> ReLU
-> Linear 128 -> LayerNorm -> ReLU
-> Linear 2 -> tanh -> [steer, accel_brake]

```

108 674 parametry, 429 KB po serializacji (torch.save(state_dict)). Ładowanie: torch.load(path, map_location="cpu", weights_only=True).

8.2 Aktor-krytyk SAC (core/custom_policy.py)

LayerNormSACPolicy — SACPolicy ze Stable-Baselines3 z LayerNorm wstawionym po każdej ukrytej warstwie Linear, zarówno w trzonie aktora, jak i w każdej z dwóch sieci Q (podwójne Q, cel = minimum). Rozmiary warstw ukrytych [256, 256, 128] dla aktora i krytyków (config.SACConfig.pi_layers/qf_layers). LayerNorm przyjęto w S2 po dywergencji krytyka i utrzymano odtąd wszędzie.

8.3 Główne hiperparametry

Parametr	SAC etap 1	Residual SAC
learning rate	3e-4	1e-4
replay buffer	1 000 000	500 000
batch size	256	256
γ / τ	0,99 / 0,005	0,99 / 0,005
entropy coef	auto (cel -2)	0,02 stałe
gSDE	wł., próbkowanie co krok	wł., próbkowanie co krok
harmonogram treningu	co krok	co epizod (gradient_steps = -1)
kotwica BC β_0 / zanik	100 / 300k kroków	100 / 500k kroków
rozgrzewka (learning_starts)	10 000	zasiew 20k kroków bazy
granice residuala δ	—	(0,15 skręt, 0,50 gaz/hamulec)

Dwie z tych wartości są krytyczne dla bezpieczeństwa, a nie kwestią preferencji: `sde_sample_freq = 1` (przytrzymanie szumu przez 8 taktów = 160 ms = 6 m drogi przy prędkości -> wypadek) oraz trening co epizod (propagacja wsteczna wewnątrz okna sterowania 20 ms rozsynchronizowuje fizykę; zob. dokument post-mortem).

9. Integracja z TORCS (Windows i Linux)

9.1 Protokół

SCR (Simulated Car Racing) przez UDP, 50 Hz. Telemetria wejściowa: 23 pola (kąt, prędkości, 19 wiązek toru, 36 wiązek przeciwników, rpm, bieg, paliwo, uszkodzenia, distFromStart/distRaced, czasy okrążeń, poślizgi kół, trackPos, z, racePos). Komendy wyjściowe: steer, accel, brake, gear, meta. Odpowiedź musi dotrzeć w takcie 20 ms, inaczej TORCS ponawia poprzednią komendę.

9.2 Windows (GUI)

1. Zainstaluj TORCS 1.3.7 + patch SCR (zob. README §Installation). Oczekiwane ścieżki (edytuj `config.py`, jeśli inne): `TORCS_EXE = D:\torcs\torcs\wtorcs.exe`, `TORCS_CONFIG_DIR = D:\torcs\torcs`.
2. Uruchom wyścig ręcznie (Race -> Practice -> New Race ze sterownikiem `scr_server`) **albo** pozwól kodowi to zrobić: `core/torcs_env_sac` uruchamia `wtorcs.exe` i steruje menu przez `autostart_win.py` (naciśnięcia klawiszy `pyautogui`; okno TORCS musi dać się aktywować).
3. Klient łączy się na porcie UDP 3001 (indeks sterownika 0).

9.3 Linux / WSL (headless)

Zbuduj TORCS 1.3.7 ze źródła `fmirus/torcs-1.3.7` (fizyka identyczna z kontenerem konkursowym) i zainstaluj sterownik serwera SCR — pełny przepis w [README_WSL.md](#). Kluczowe fakty:

- `torcs -r <race.xml> z display mode = results only` działa bez renderowania 3D: ograniczony przez CPU, może działać szybciej niż w czasie rzeczywistym.
- `search/optuna_teacher_linux.py::install_practice_xml(port)` kopiuje `practice.xml` z roota repo do `~/torcs/config/raceman/` i podmienia indeks sterownika SCR (= port - 3001) per worker.
- Każdy równoległy worker potrzebuje **własnego HOME** (`HOME=~/torcs_w<k>`), żeby stan `~/torcs` się nie kolidował.
- Pierwsze uruchomienie na zimnym HOME segfaultuje; `TorcsSACEnv` wykonuje jednorazową rozgrzewkę (`timeout 10 torcs`) i ponawia połączenia (do 4x).
- Skompilowany serwer SCR udostępnia **10 slotów robotów** (porty 3001–3010) — twardy limit równoległości na maszynie.
- Farma WSL startuje z `-nofuel -nodamage -nolaptime`; ścieżka zgłoszenia działa **z** paliwem i uszkodzeniami. Spodziewaj się różnicy $\approx 0,7$ s w czasie okrążenia; podawaj liczby ze ścieżki zgłoszenia jako główne.

9.4 Konfiguracja wyścigu

practice.xml (root repo): Corkscrew, kategoria road, sesja practice, 1 sterownik (scr_server idx 0), start zatrzymany. Jest *kopiowany*, nigdy czytany w miejscu — ale źródło kopiowania rozwiązywane jest względem roota repo, więc plik nie może się przenieść.

10. Pipeline'y treningowe z komendami

Wszystkie komendy uruchamiasz z **katalogu głównego repo**. Windows: użyj python; WSL: python3 w venv. Pełny opisany przepis: [DISTILL_COMMANDS.md](#).

10.0 Zgłoszenie (odtworzenie punktowanego okrążenia)

core/submit_agent.py uruchamia jedną z dwóch wybieralnych polityk, obie przez to samo przetwarzanie driving_aids i obie osiągające okrążenie ~92 s ze startu zatrzymanego — deterministyczny deliverable BC/DAGger (92,04 s) oraz politykę residualną SAC, która dokłada korekcje uczenia ze wzmocnieniem na wierzchu:

```
# BC – punktowany deliverable 92.04 s (domyślnie):
python -m core.submit_agent --weights checkpoints/bc_v6.pth --episodes 1 --port 3001

# Residual – korekcje SAC na zamrożonej bazie: clip(base +  $\delta \cdot \text{residual}$ ).
# Dołączony residual był trenowany na bc_v6.pth (domyślny --base):
python -m core.submit_agent --residual checkpoints/residual_sac_latest.zip --episodes 1 --port 3001
```

Ścieżka residualna ładuje .zip SAC (SB3) z custom_objects={"policy_class": LayerNormSACPolicy} (co neutralizuje zmianę ścieżki modułów po przepakowaniu), czyta δ z Config.residual (delta_steer=0.15, delta_accel=0.50) i łączy je z zamrożoną siecią --base dokładnie tak, jak training/residual_env.py::compute_final --base musi odpowiadać sieci, na której residual był trenowany (dołączony residual_sac_latest.zip trenowano na bc_v6.pth, wartość domyślna). Przy inferencji nie jest potrzebny VecNormalize (ścieżka ewaluacji residuala używa przezroczystego VecNormalize(norm_obs=False, norm_reward=False)). Bufor odtwarzania 138 MB tego checkpointu nie jest dołączany — tylko residual_sac_latest.zip.

10.1 Przeszukiwanie teachera (farma WSL)

```
# jednorazowo: utwórz study + zakolejkuj zasiewy
python -m search.new_study_v6 --study-name teacher_v6_ow1 \
  --storage sqlite:///home/user/optuna_ow1.db

# per worker k = 0..9
HOME=~/.torcs_w$k python3 -m search.optuna_teacher_v6_linux \
  --study-name teacher_v6_ow1 \
  --storage sqlite:///home/user/optuna_ow1.db \
  --port $((3001+k)) --n-trials 1000000

# dashboard na żywo / lustro w chmurze (opcjonalne)
python -m search.monitor
python -m search.sync_to_cloud

# eksport najlepszego trialu
python -m search.export_teacher_v6 --study-name teacher_v6_ow1 \
  --storage sqlite:///home/user/optuna_ow1.db \
  --output checkpoints/best_teacher_v6.json
```

10.2 Destylacja (BC -> DAgger)

```
# smoke test (3k kroków – spodziewaj się okrążenia ~88 s w minutę)
python -m agents.bc_pretrain --controller v6 \
--teacher-params checkpoints/best_teacher_v6.json \
--n-steps 3000 --output checkpoints/_bc_smoke.pth

# pełny BC (500k przejść)
python -m agents.bc_pretrain --controller v6 \
--teacher-params checkpoints/best_teacher_v6.json \
--n-steps 500000 --output checkpoints/bc_v6.pth \
--prev-steer-noise 0.15

# doszlifowanie DAgger (5 × 100k, zasiane teacherem)
python -m agents.dagger --controller v6 \
--teacher-params checkpoints/best_teacher_v6.json \
--bc-weights checkpoints/bc_v6.pth \
--iterations 5 --steps-per-iter 100000 --epochs 30 \
--output checkpoints/dagger_v6.pth

# validacja
python -m tools.eval_policy --bc-pretrain checkpoints/bc_v6.pth --episodes 3
```

Nie uruchamiaj zbierania BC/DAgger na maszynie z żywymi workerami Optuny: zbieranie używa portu 3001 i zawężonego ubijania TORCS-a przy resecie. Zatrzymaj workery albo użyj innej maszyny.

10.3 Residual SAC (opcjonalne doszlifowanie)

```
# świeży przebieg względem danej zamrożonej bazy (--resume none jest obowiązkowe
# zawsze, gdy zmieniła się sieć bazowa)
python -m training.train_residual_sac \
--dagger-weights checkpoints/bc_v6.pth \
--resume none --skip-floor-check --ent-coef-final 0.005

# ewaluacja checkpointu residuala względem jego bazy
python -m training.train_residual_sac \
--eval checkpoints/residual_sac_<N>_steps.zip \
--dagger-weights checkpoints/bc_v6.pth # szukaj "IMPROVED on DAgger"
```

10.4 SAC od zera (linia baseline'ów)

```
python -m training.train_sac --stage 1 --timesteps 5000000 \
--seed-demos 20000 --bc-pretrain checkpoints/bc_v6.pth
```

10.5 Etap 2 (multi-car, niepunktowany)

```
python -m phase2.train_mass_start # zob. phase2/README.md
python -m phase2.submit_agent_stage2 --weights <stage2 ckpt>
```

11. Rozproszona infrastruktura Optuny

- **Storage:** `search/optuna_teacher_v3.py::make_storage()` buduje albo storage SQLite w trybie WAL (busy_timeout, skip_compatibility_check), albo pulowany storage PostgreSQL przypięty do

pool_size=1, max_overflow=0 na proces (poprawka na wyczerpanie puli w chmurze).

- **Użyta topologia:** każda maszyna prowadziła workery względem **lokalnego SQLite**; search/sync_to_cloud.py lustrzył ukończone triale jednokierunkowo do chmurowego study PostgreSQL (pomijając pojedyncze triale zasiewowe spoza rozkładu zamiast przerywać całą partię).
- **Osiągnięta skala:** ~15 000 triali w różnych study (v3: 2 210 -> plateau 118,8 s; v5b/v6 zbiegły do 91,2 s / ~88,8 s przy 10 workerach/maszynie).
- **Anatomia trialu:** ask params -> install_practice_xml(port) -> headless torcs -r -> teacher jedzie 1–3 okrążenia przez UDP -> zgłoszenie najlepszego okrążenia -> tell(study). Zaklinowane, ale na torze auta ucina bailout braku postępu (~4 s bez ruchu do przodu).
- **Naprawione tryby awarii** (szczegóły w EXPERIMENT_LOG.md §2, §7, §10): globalny pkill torcs w samonaprawie klienta kaskadujący między workerami (teraz zawężony per worker); workery cicho kończące na --n-trials study; 300-sekundowe timeouty triali od zaklinowanych aut.

12. Checkpointy i artefakty

Plik	Format	Zawartość
checkpoints/bc_v6.pth	PyTorch state dict	DELIVERABLE — zdestylowana polityka; okrążenie punktowane 92,04 s
checkpoints/dagger_policy_v2.pth	PyTorch state dict	polityka zapasowa (106,96 s, 6/6 czystych epizodów)
checkpoints/best_teacher_v6.json	JSON (54 param.)	zamrożony nastrojony teacher — pełna proveniencja/reprodukcja
checkpoints/best_teacher_v2_pg18s.json	JSON	historyczny snapshot teachera
*_steps.zip + *vecnorm*.pkl (gitignored)	SB3	checkpointy treningu SAC (polityka+optymalizator / statystyki VecNormalize)

Ładowanie deliverable'a:

```
from agents.bc_pretrain import BCNetwork
import torch

net = BCNetwork(obs_dim=32, action_dim=2, hidden_sizes=[256, 256, 128])
net.load_state_dict(torch.load("checkpoints/bc_v6.pth",
    map_location="cpu", weights_only=True))
net.eval()
action = net(obs_tensor) # [steer, accel_brake] 2 [-1, 1]2
```

(Dawny checkpoint 31-wym. jest automatycznie dopełniany zerową kolumną prev_steer przez training/residual_env.py::_load_dagger.)

13. Referencja konfiguracji

Wszystko żyje w config.py jako dataclassy pod singletonem Config:

Pole	Zawartość	Wartości krytyczne
Config.torcs	ścieżki, porty, tor, progi terminacji	track_length_m=3602, off-track_trackpos_threshold=1.10, max_steps_per_episode=9000, base_port=3001
Config.observation (alias obs)	stałe normalizacji, wymiary	stage1_dim=32, stage2_dim=68, gear_max=7.0

Pole	Zawartość	Wartości krytyczne
Config.aids (alias action)	parametry biegów/TCS/startu	rpm_upshift=17800, rpm_downshift=9000, max_gear=6 (clamp SCR), tcs_slip_threshold=5.0
Config.reward	wszystkie wagi nagrody z \$6	lap_bonus=500, lap_target_time=90
Config.sac Config.residual	sieci SAC + hiperparametry residual RL	pi_layers=qf_layers=[256,256,128] delta_steel=0.15, delta_accel=0.50, train_freq=(1,"episode"), gradient_steps=-1
Config.teacher_v3	domyślne teachera waypointowego v3	12 prędkości + 12 waypointów linii
Config.multi	multi-instance Windows	n_instances=6, base_port=3001
Config.training / Config.curriculum	harmonogramy, zasiew, rampa etapu 2	seed_steps=50000

TempConfig({...}) to menedżer kontekstu, który tymczasowo nadpisuje kropkowane ścieżki konfiguracji na czas trialu Optuny.

14. Ograniczenia i pułapki

1. **Uruchamiaj punkty wejścia z roota repo** (python -m pakiet.modul). config.py to moduł na poziomie roota: każdy proces, którego sys.path[0] to root repo, rozwiąże import config; uruchomienie pliku z wnętrza katalogu pakietu nie zadziała.
2. **practice.xml musi zostać w rootcie repo**. search/optuna_teacher_linux.py rozwiązuje go jeden katalog wyżej niż własne położenie i kopiuje do ~/.torcs przy każdym uruchomieniu headless.
3. **autostart_win.py musi zostać w rootcie repo**. Jest wywoływany po bezwzględnej ścieżce roota przez core/torcs_env_sac.py / core/snakeoil3_gym.py oraz przez training/multi_instance_torcs.py.
4. **Terminacja poza torem używa tylko |trackPos| > 1.10** (§7). Nie „utwardzaj” jej sprawdzaniem znaku dalmierzy.
5. **--resume auto w treningu residualnym ładuje najnowszy checkpoint z dysku**. Po zmianie sieci bazowej podaj --resume none albo usuń stare checkpointy residual_sac*.
6. **HOME per worker w WSL** (§9.3) i **limit 10 robotów SCR** to twarde ograniczenia operacyjne.
7. **Parzystość flag Windows vs WSL**: farma działała z -nofuel -nodamage; zgłoszenie działa z oboma włączonymi (~ 0,7 s wolniej). Podawaj liczby ze ścieżki zgłoszenia.
8. **Sekrety**: DSN PostgreSQL żyje w nietrackowanym pliku .pg_url (w gitignore). Nigdy go nie commituj; search/sync_to_cloud.py oczekuje go w rootcie repo, gdy używane jest lustro w chmurze.
9. **Nie propaguj wstecznie wewnątrz pętli sterowania**. Jeśli zmieniasz harmonogramy treningu, trzymaj aktualizacje gradientu poza ścieżką czasu rzeczywistego 20 ms (train_freq=(1,"episode")); sygnaturę awarii opisuje dokument post-mortem.

15. Rozwiązywanie problemów

Objaw	Prawdopodobna przyczyna -> poprawka
ModuleNotFoundError: config	Punkt wejścia nie wystartowany z roota repo. cd do roota i użyj python -m pakiet.modul.
Klient wisi na Waiting for server on 3001	TORCS nie działa / brak sterownika SCR w wyścigu / zły port. Uruchom Practice ze scr_server idx 0 albo sprawdź --port.

Objaw	Prawdopodobna przyczyna -> poprawka
torcs -r segfaultuje przy pierwszym starcie WSL	Zimny ~/ .torcs. Uruchom jednorazową rozgrzewkę (timeout 10 torcs) albo po prostu ponów — TorcsSACEnv robi oba automatycznie.
Trial Optuny zwracają 999 / brak okrążeń	Parametry teachera poza zakresem albo instancja TORCS martwa. Uruchom jedno okrążenie ręcznie: python -m tools.diag_lap --port 3001.
Workery cicho kończą po N trialach	Study osiągnęło swój --n-trials. Uruchom ponownie z --n-trials 1000000.
max clients reached ... pool_size (PostgreSQL)	Za dużo pulowanych połączeń. Użyj make_storage() (przypina pulę do 1/proces) albo przenieś workery na lokalny SQLite + sync_to_cloud.
Każdy epizod BC kończy się przy ~2 463 m	Klauzula min(track) < 0 została przywrócona do terminacji. Usuń ją (§7, §14.4).
Świetny loss walidacyjny BC, auto rozbija się w pętli zamkniętej	Skrót copycat na prev_steer. Trenuj z --prev-steer-noise 0.15 (domyślne).
Auto hamuje za późno i rozbija się tylko w trybie treningu	Aktualizacje gradientu na ścieżce sterowania. Przywróć train_freq=(1,"episode"), gradient_steps=-1.
Polityka residualna od razu gorsza niż jej baza	Wznowiono checkpoint trenowany na innej bazie. --resume none.
Prędkość maksymalna utknęła ≈ 174 km/h	Regresja harmonogramu biegów. Sprawdź Config.aids: rpm_upshift=17800, max_gear=6 (oraz gear_max=7.0 — stała normalizacji treningowej — w konfiguracji obserwacji).
Okno TORCS nie startuje wyścigu automatycznie (Windows)	autostart_win.py potrzebuje okna TORCS dającego się aktywować; nie blokuj ekranu, sprawdź instalację pyautogui/pygetwindow.